About this PDF

Built only from: **Artificial Intelligence (Problem Solving and Search)** — https://nptel.ac.in/courses/106105077

Licence: CC BY-SA 4.0. Accessed .

Covers 7 of 7 syllabus sub-topics of this unit.

Problem solving agents

Definition

A problem-solving agent is a goal-based agent that decides what actions to take by finding sequences of actions that lead to desirable states.

Explanation

The agent follows a four-phase cycle:

1. **Goal formulation** - defining the objective based on the current situation and performance measure.
2. **Problem formulation** - deciding what actions and states to consider.
3. **Search** - simulating sequences of actions in an internal model to find a path to the goal.
4. **Execution** - performing the actions in the physical world.

A well-defined search problem consists of five formal components: initial state, actions, transition model, goal test, and path cost. The search process builds a search tree with the initial state as the root, branches representing actions, and nodes representing states reached. The frontier (or open list) stores all leaf nodes available for expansion. Search algorithms are evaluated across four criteria: completeness, time complexity, space complexity, and optimality.

Algorithm / Steps

1. **Goal formulation** - define the objective based on the current situation and performance measure.
2. **Problem formulation** - decide what actions and states to consider.
3. **Search** - simulate sequences of actions in an internal model to find a path to the goal.
4. **Execution** - perform the actions in the physical world.

Key points

- A problem-solving agent is goal-based and finds action sequences that lead to desirable states.
- It operates through a four-phase cycle: goal formulation, problem formulation, search, and execution.
- A search problem is defined by initial state, actions, transition model, goal test, and path cost.

- The search process builds a search tree; the frontier holds leaf nodes for expansion.
- Search algorithms are judged by completeness, time complexity, space complexity, and optimality.

search algorithms

A search problem in artificial intelligence is formally formulated by defining the state space, which consists of the set of all states reachable from the initial state through legal action sequences. The search process systematically constructs a search tree where the root node represents the initial state, directed branches represent available actions, and child nodes represent resulting states.

During search execution, candidate leaf nodes available for expansion are stored in a data structure designated as the frontier or open list. Search algorithms select a node from the frontier based on a specific expansion strategy and apply the transition model to generate its successors.

Search algorithms are rigorously assessed and compared using four standardized performance metrics:

- Completeness: Guarantees finding a solution whenever at least one valid solution exists in the state space.
- Time Complexity: The total number of nodes generated or computational operations required before reaching a goal state.
- Space Complexity: The maximum amount of working memory or number of nodes stored simultaneously during search.
- Optimality: The ability to return a solution path that achieves the lowest possible path cost among all alternative solutions.

Key structural parameters governing complexity include the branching factor b representing maximum successors per node, the depth d of the shallowest goal state, and maximum path length m in the state space.

uninformed search strategies

Definition

Uninformed search strategies, also known as blind search, are search algorithms that explore the search space without using domain-specific knowledge beyond the problem definition. They rely on generic operations such as node expansion and path-cost evaluation.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

Explanation

These strategies expand nodes according to a specific order. Breadth-First Search expands the shallowest unexpanded node first using a FIFO queue, guaranteeing completeness when the branching factor is finite and optimality when all step costs are equal. Uniform-Cost Search expands the node with the lowest cumulative path cost using a priority queue; it is complete if step costs exceed a small positive constant and optimal for arbitrary non-negative step costs. Depth-First Search expands the deepest unexpanded node first using a LIFO stack; it is memory efficient but not complete in infinite or cyclic spaces and not optimal. Depth-Limited Search limits DFS to a fixed depth, while Iterative Deepening Search repeatedly applies Depth-Limited Search with increasing limits, combining the completeness and optimality of BFS with the linear space of DFS. Bidirectional Search runs two simultaneous searches from the initial and goal states, halving the effective depth and reducing time and space to O(b^(d/2)) when a goal state is explicit and predecessors can be computed efficiently.

Key points

- Breadth-First Search is complete for finite branching factors and optimal when step costs are equal; it uses a FIFO queue and has time and space complexity O(b^d).
- Uniform-Cost Search is complete if step costs exceed a small positive constant and optimal for arbitrary non-negative step costs; it uses a priority queue and has time and space complexity O(b^(1+floor(C*/epsilon))).
- Depth-First Search is memory efficient with space O(b·m) but is not complete in infinite or cyclic spaces and is not optimal.
- Depth-Limited Search limits DFS to a depth l; Iterative Deepening Search runs DLS repeatedly with increasing limits, achieving completeness and optimality with space O(b·d).
- Bidirectional Search reduces time and space to O(b^(d/2)) when an explicit goal state exists and predecessors can be efficiently computed.

Heuristic search strategies

Definition

An informed search algorithm uses a heuristic function h(n) that estimates the cheapest cost from node n to a goal state; the heuristic is zero for a goal node.

Explanation

Greedy Best-First Search expands the node with the lowest h(n) using a priority queue; it is incomplete in infinite spaces and not optimal. A* evaluates nodes with f(n)=g(n)+h(n) and is optimal when the heuristic is admissible; consistency guarantees optimality in graph search. Memory-bounded variants such as Recursive Best-First Search and SMA* mimic best-first search within linear space by pruning and regenerating nodes. Local search methods such as hill-climbing, simulated annealing, and genetic algorithms use heuristic evaluation to guide moves without exploring all paths.

Example

The 8-Queens puzzle is a classic example where a local search algorithm evaluates board configurations to reach a goal state with no attacking queens.

Algorithm / Steps

1. Greedy Best-First Search: Expand the node with the lowest h(n) using a priority queue.
2. A* Search: Compute f(n)=g(n)+h(n) for each node and expand the node with the lowest f(n).
3. Simulated Annealing:
    1. Select a random neighbor of the current state.
    2. If the neighbor improves the evaluation function, accept it unconditionally.
    3. Decrease the temperature T according to a cooling schedule.
4. Genetic Algorithm:
    1. Select candidate solutions based on a fitness function.
    2. Perform crossover between selected parents to produce offspring.
    3. Apply mutation by randomly altering symbols with a low probability.

Key points

- Heuristic functions estimate the cheapest cost to a goal and are zero for goal nodes.
- Greedy Best-First Search uses only h(n) and is not optimal.
- A* Search uses f(n)=g(n)+h(n) and is optimal if the heuristic is admissible; consistency ensures optimality in graph search.
- Memory-bounded heuristic search (RBFS, SMA*) manages strict memory limits by pruning and regenerating nodes.
- Local search variants (hill-climbing, simulated annealing, genetic algorithms) use heuristic evaluation to guide moves without exhaustive search.

Local search and optimization problems

Definition

Local search algorithms operate on pure state formulations where the path to the goal is irrelevant, and only the final configuration matters. They maintain a single current state and move only to neighboring states.

Explanation

Hill-climbing is an iterative greedy algorithm that continuously moves in the direction of increasing value (or decreasing cost). It terminates when it reaches a peak where no neighbor has a higher value. Key vulnerabilities include local maxima (suboptimal peaks higher than neighbors but lower than the global maximum), plateaus (flat areas of the state-space landscape where evaluation is constant), and ridges (sequences of local peaks where progress requires simultaneous coordinate changes).

AcademicsGen — source: NPTEL — CC BY-SA 4.0

Example

The 8-Queens puzzle, VLSI circuit layout, or travelling-salesperson problem are typical optimization contexts for local search.

Algorithm / Steps

1. **Simulated Annealing** - Select a random neighbor. If the neighbor improves the evaluation function, accept it unconditionally. If the neighbor worsens the state by DeltaE, accept it with probability exp(DeltaE / T), where T is a temperature parameter that gradually decreases according to an annealing cooling schedule. This mechanism allows probabilistic escape from local maxima.
2. **Genetic Algorithms** - a. Selection: sample candidate solutions based on a fitness function. b. Crossover: recombine pairs of parents at random crossover points to produce offspring. c. Mutation: randomly alter symbols with a low mutation probability.

Key points

- Local search focuses on the final configuration, not the path to it.
- Hill-climbing can get stuck at local maxima, plateaus, or ridges.
- Variants such as stochastic hill-climbing, first-choice hill-climbing, and random-restart hill-climbing help mitigate these issues.
- Simulated annealing introduces probabilistic acceptance of worse states to escape local optima.
- Genetic algorithms use a population of solutions and evolutionary operators (selection, crossover, mutation) to explore the search space.

adversarial search

Definition

Adversarial search refers to the process of solving multi-agent game problems where two or more players act in opposition. The game is represented as a deterministic, fully observable, turn-taking, zero-sum environment with a formal state space defined by an initial state s0, a function TO-MOVE(s) that indicates whose turn it is, ACTIONS(s) that lists legal actions, RESULT(s,a) that gives the successor state, TERMINAL-TEST(s) that detects the end of the game, and UTILITY(s,p) that returns the numerical payoff for player p.

Explanation

The Minimax algorithm computes the optimal strategy for the maximizing player (MAX) under the assumption that the minimizing player (MIN) also plays optimally. MAX chooses actions that maximize the utility value, while MIN chooses actions that minimize it. The value of a terminal state is its utility, and this value is propagated upward through the game tree. Minimax is complete on finite game trees, guarantees optimal play against an optimal opponent, and has time complexity O(b^m) and space complexity O(b*m). Alpha-Beta pruning improves efficiency by eliminating subtrees that cannot affect the final decision. Two bounds are maintained: alpha, the best value found so far for MAX along the current path, and beta, the best value found so far for MIN. A branch is pruned whenever alpha)= beta, which can reduce the effective branching factor and allow deeper search.

Key points

- Games are modeled as deterministic, fully observable, turn-taking, zero-sum problems with formal state-space components.
- Minimax computes optimal strategies for MAX and MIN, with complexity O(b^m) time and O(b*m) space.
- Alpha-Beta pruning discards branches where alpha)= beta, potentially reducing time complexity to O(b^(m/2)) with optimal move ordering.

constraint satisfaction problems (CSP)

Definition

A Constraint Satisfaction Problem consists of three core components: variables, domains, and constraints. A state is an assignment of values to variables. An assignment is consistent if it does not violate any constraints, and complete if every variable is assigned. A solution is a consistent, complete assignment.

Explanation

CSP solvers use constraint propagation to eliminate values from variable domains. A directed arc from variable Xi to Xj is arc-consistent if for every value in Di there exists an allowable value in Dj that satisfies the binary constraint between Xi and Xj. The AC-3 algorithm enforces arc consistency across all edges in O(c × d^3) time, where c is the number of binary constraints and d is the maximum domain size.

Algorithm / Steps

1. Perform standard depth-first search assigning single variables sequentially; dead ends trigger backtracking.
2. Apply Minimum Remaining Values (MRV) heuristic to choose the variable with the fewest legal values remaining.
3. Use Degree Heuristic to break ties by selecting the variable with the most constraints on remaining unassigned variables.
4. Prefer Least Constraining Value (LCV) to choose a value that rules out the fewest choices for neighboring variables.

5. Employ Forward Checking to track remaining legal values for unassigned variables and terminate search if any domain becomes empty.

Key points

- CSPs are defined by variables, domains, and constraints; a solution is a consistent, complete assignment.
- Constraint propagation via AC-3 reduces domains in O(c × d^3) time.
- Backtracking search is enhanced by MRV, Degree, LCV, and Forward Checking heuristics.

Source and licence

This PDF is adapted from **Artificial Intelligence (Problem Solving and Search)** (https://nptel.ac.in/courses/106105077), accessed .

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).